

Classe 4 · Informatica (Laboratorio Professionale) · 29/09/2026

Git, versioning e Intelligenza Artificiale

Per gli allievi (lavori anonimi)

Teoria, compito assegnato e i sei lavori consegnati dagli allievi. I lavori sono riportati in forma anonima, per confronto e ripasso.

Teoria

1. I tre strumenti e come si legano

1. Usiamo tre ambienti diversi, ognuno con un compito preciso:
2. **Git e GitHub**: il magazzino ordinato del lavoro, dove ogni versione resta salvata e condivisibile.
3. **Gemini**: l'IA (Intelligenza Artificiale) generativa a cui si parla; le si da un testo (anche un file .md) e ti spiega, traduce, riassume.
4. **NotebookLM**: un tutor che risponde SOLO sui documenti che carichi (per esempio un PDF), quindi resta fedele alla fonte.
5. Idea da tenere a mente: **Git conserva, Gemini spiega, NotebookLM ripassa sulla fonte**. Non sono concorrenti: si usano insieme.

2. Git e GitHub

1. **Git** è un sistema di **versionamento**: tiene la storia del lavoro, così ogni salvataggio importante resta e si può tornare indietro.
2. **GitHub** è il sito in rete dove il lavoro versionato vive e si condivide: è "Git su internet". Attenzione a non confonderli: **Git** è lo strumento (il motore), **GitHub** è il servizio online (il garage online).

2.1 GitHub: un'azienda e un servizio

1. È un'**azienda** (GitHub, Inc., dal 2018 di Microsoft): come ogni azienda deve mantenersi.
2. È un **servizio** online: un sito dove metti i progetti in rete, li tieni al sicuro e ci lavori in gruppo, tutto da browser.

2.2 Il repository, il commit, il branch

1. Un **repository** (o "repo") è la cartella-progetto con dentro i file e tutta la loro storia.
2. Un **commit** è un salvataggio con etichetta: fotografa il progetto in quel momento e gli mette un messaggio ("cosa ho cambiato"). Tanti commit in fila fanno la **storia**.
3. Un **branch** (ramo) è una strada di lavoro parallela, per provare senza rovinare la versione principale (il **main**); riunire un ramo al main si chiama **merge**.
4. La **Pull Request** (PR) è la proposta di unire il proprio ramo nella versione principale, dopo che qualcuno l'ha controllata.

2.3 Che tipo di servizio: PaaS e Freemium

1. **PaaS** (Platform as a Service: piattaforma come servizio): qualcuno ti mette online una piattaforma già pronta e tu la usi senza costruirti server e computer. Fa parte della famiglia "...aaS" (IaaS = server nudi, PaaS = piattaforma pronta, SaaS = software pronto nel browser).
2. **Freemium** (free + premium): le funzioni di base sono gratis; per le funzioni avanzate si paga. A te studente basta la parte gratis.

3. Il versioning: le versioni 1.2.3

1. Quando un progetto raggiunge un punto "buono", gli si dà un numero di versione di tre cifre 1.2.3:
2. **major** (la prima cifra): aumenta quando cambi tutto in modo grosso, magari incompatibile col vecchio.
3. **minor** (la seconda): aumenta quando aggiungi una funzione nuova, ma il resto funziona ancora.
4. **patch** (la terza): aumenta quando correggi solo un bug, niente di nuovo.
5. Esempio: v1.0.0 prima versione giocabile, aggiungi i suoni v1.1.0, correggi un bug v1.1.1, rifai tutto da capo v2.0.0.
6. Una versione pubblicata (**release**) è "congelata".

Regola d'oro del versioning: una versione già pubblicata non si riusa mai. Se cambi anche una sola cosa, prima aumenti il numero e poi pubblichi.

4. Casi d'uso di Git (i grafi, descritti a parole)

1. **Sviluppo lineare**: un solo autore, nessun ramo. I commit vanno in fila, la storia è una linea dritta. Se qualcosa si rompe, torni indietro di un commit.
2. **Una funzione su un branch, poi merge**: provi una funzione su un ramo separato mentre il main resta sano; quando è pronta la unisci (merge).
3. **Due funzioni in parallelo**: due rami partono insieme e si fondono in tempi diversi; il main raccoglie i pezzi man mano.
4. **Hotfix urgente**: durante una funzione lunga esce un bug grave; apri un ramo hotfix corto, lo correggi e lo unisci subito, poi torni alla tua funzione.
5. **Le release**: ogni tanto pubblichi una versione (bandierina sul main): v1.0, v1.1, v1.2... fino a un cambiamento grosso v2.0. Ogni release è congelata.
6. **Ramo abbandonato**: provi un'idea su un ramo e non la unisci mai; il main non ne risente. Nessun danno.
7. **Ramo da un altro ramo**: una funzione grande stacca un sotto-ramo; prima rientra il sotto-ramo nel ramo, poi il ramo nel main (scatole dentro scatole).
8. **Grande integrazione (gruppo)**: ognuno ha il suo ramo (grafica, movimenti, audio, menu) e alla fine confluiscono tutti nel main con le Pull Request: nasce la v1.0.
9. **Ramo di manutenzione**: il main va verso la v2.0, ma per chi usa ancora la v1 tieni un ramo parallelo con solo i fix (v1.1, v1.2).
10. **Revert**: un commit ha introdotto un problema; non cancelli la storia, aggiungi un nuovo commit che annulla quello sbagliato (revert), lasciando traccia onesta.
11. **Conflitto di merge**: due rami cambiano la stessa riga in modo diverso; Git si ferma e una persona sceglie cosa tenere, chiudendo con un commit di risoluzione.
12. **Squadra a 5 rami con due release**: sul main si pubblicano v1.0 e v2.0 mentre quattro rami lavorano in parallelo e rientrano in momenti diversi.

13. **Rami annidati a 5 livelli**: un lavoro grande si divide in rami dentro rami, fino a cinque livelli; ogni livello rientra nel precedente, come le matriosche.
14. **Due major insieme**: dopo la v1.0 il main punta alla v2.0, ma restano vivi due rami di manutenzione (v1.x e poi v2.x).
15. **Storia realistica**: tutto insieme — release, feature con sotto-ramo, hotfix, revert e un merge con conflitto: la fotografia di un progetto vero.

Come si legge un grafo: ogni pallino è un commit, la linea orizzontale in alto è il main, le linee colorate che si staccano sono branch, quando due linee si riuniscono è un merge, le bandierine gialle sono le release.

5. I file Markdown (.md)

1. Un file **Markdown** (estensione **.md**) è **testo puro** con pochi simboli:
2. **#** a inizio riga fa un titolo;
3. ****parola**** mette in grassetto;
4. una riga con **1.** fa un elenco numerato.
5. Si legge ovunque e non contiene immagini pesanti: solo contenuto. Per questo l'IA lo legge molto bene: nel corso diamo le dispense anche in .md proprio per darle a Gemini.

6. Gemini: l'IA generativa, e come usarla con un file .md

1. **Gemini** è un'IA **generativa**: le scrivi una richiesta (un "prompt") e genera una risposta: spiega, traduce, riassume, fa domande di ripasso.
2. L'IA può **sbagliare o inventare** (allucinazioni): quello che dice va controllato, soprattutto date, numeri e fonti.

6.1 Passi per usare Gemini con un file .md

1. Nel browser vai su questo indirizzo, con l'account della scuola:

`gemini.google.com`

2. Apri il file .md, **seleziona tutto** e **copialo**; torna su Gemini e **incolla** (oppure allega il file).
3. Scrivi la richiesta, per esempio:

`Spiegami con parole semplici la parte che non capisco: [incolla la frase].`

6.2 Quale versione di Gemini scegliere

1. In alto Gemini ha un menu per la **versione del modello**. I nomi/numeri cambiano; conta a cosa serve ciascuna:
2. **Flash** (veloce): risposte rapide per cose semplici (spiegazioni, traduzioni, riassunti brevi). La scelta di tutti i giorni.

3. **Pro** (per esempio "3.1 Pro", piu potente): per compiti complessi e testi lunghi. Piu lenta ma piu capace.
4. **Ragionamento** (per esempio "3.6 ragionamento", che "pensa" prima di rispondere): per problemi difficili, passo per passo. La piu lenta ma la piu accurata.
5. Regola pratica: parti da **Flash**; se la risposta e complessa o ti sembra sbagliata, passa a **Pro** o **Ragionamento** e richiedi.
6. Guarda a **cosa serve** la versione (veloce / potente / ragiona), non solo al numero.

Le versioni piu potenti o di "ragionamento", con l'account gratuito, possono avere un limite giornaliero: usale quando servono davvero, per il resto va bene Flash.

7. NotebookLM: il tutor sulle fonti, e come usarlo con un PDF

1. **NotebookLM** risponde **solo** sui documenti che carichi: se carichi un PDF, ti risponde usando quel PDF e non inventa fuori da li.
2. E ottimo per **studiare e ripassare** su una fonte precisa (una dispensa, un capitolo): resta fedele al testo.

7.1 Passi per usare NotebookLM con un PDF

1. Nel browser vai su questo indirizzo, con l'account della scuola:

`notebooklm.google.com`

2. Clicca **Crea nuovo**, poi **Carica**, e scegli un file **PDF**; aspetta che la fonte compaia a sinistra.
3. Nel riquadro della chat scrivi, per esempio:

Fammi un riassunto semplice di questo documento.

Se non hai un PDF sul computer: su github.com entra nella cartella del documento, clicca sul PDF e poi sull'icona di download (freccia in giu) in alto a destra.

8. Il flusso completo: i tre ambienti insieme

1. Il materiale del corso sta su **Git/GitHub** (versionato).
2. Da li si **scarica** il PDF (per leggere/stampare) o il file **.md** (per l'IA).
3. Il **.md** si da a **Gemini** per farsi spiegare o tradurre.
4. Il **PDF** si carica su **NotebookLM** per interrogarlo e ripassare restando sulla fonte.
5. Il lavoro finito si **consegna su Classroom**.

Strumento	A cosa serve	Cosa gli dai
Git / GitHub	Conservare e condividere il lavoro versionato	I file del progetto

Strumento	A cosa serve	Cosa gli dai
Gemini	Spiegare, tradurre, riassumere	Un testo o un file .md
NotebookLM	Ripassare restando fedele alla fonte	Un documento (per esempio un PDF)

9. Uso responsabile dell'IA

1. **Capire, non copiare:** l'IA è un aiuto per imparare, non un modo per saltare il lavoro. La prova del nove: saperlo spiegare a parole proprie.
2. **Controllare sempre:** l'IA può sbagliare o inventare; verifica quello che dice.
3. **Privacy e dati:** non inserire dati personali, di compagni o di famiglia negli assistenti.

10. Prendi appunti a mano

1. Prima di usare gli strumenti, disegna sul quaderno tre riquadri (Git, Gemini, NotebookLM) e, per ognuno, a cosa serve e cosa gli dai.
 2. Gli appunti a mano confluiscono nel tuo quaderno personale: aiutano a fissare i concetti e servono per la prova del nove.
-

Il compito assegnato

1. Come si consegna

1. Scrivi le risposte in un documento di **Google Documenti** (uno solo per tutto il compito).
2. Intitola il documento con **Cognome e Nome** e "**Compito Git e AI**".
3. Dove chiesto, incolla uno **screenshot** (cattura schermo) come prova.
4. Alla fine **consegna** il documento nel compito su **Classroom**.

Come si fa uno screenshot (Windows 10 e 11): premi insieme i tasti **Windows** + **Maiusc** + **S**, seleziona la parte di schermo, poi incolla nel documento con **Ctrl** + **V**.

2. Parte 1 — Git e GitHub

2.1 Domande (rispondi a parole tue)

1. Con parole tue, che differenza c'è tra **Git** e **GitHub**?
2. Che cos'è un **repository**?
3. Che cos'è un **commit** e a cosa serve avere la storia dei commit?
4. A cosa serve un **branch** e cosa propone una **Pull Request**?

2.2 Azione pratica

1. Nel browser entra su questo indirizzo con il tuo account:

`github.com`

2. Apri un **repository** del corso (quello indicato dal docente) e guarda l'elenco dei file.
3. Apri la sezione dei **commit** (la storia) e osserva gli ultimi salvataggi.
4. Fai uno **screenshot** della storia dei commit e incollalo nel documento.
5. Scrivi due righe: **cosa** mostra quella schermata (che cosa è cambiato nell'ultimo commit).

3. Parte 2 — Gemini con un file .md

3.1 Esercitazione

1. Nel browser entra su questo indirizzo con l'account della scuola:

`gemini.google.com`

2. In alto scegli la versione **Flash** (quella veloce): per questo compito basta.
3. Apri il file **.md** indicato dal docente (per esempio la dispensa di teoria in formato .md), **seleziona tutto** e **copialo**.
4. **Incolla** il testo in Gemini e chiedi, con parole tue, di spiegarti una parte. Puoi partire da qui:

Spiegami con parole semplici cosa sono i file Markdown e perché sono comodi per l'IA.

5. Poi chiedi a Gemini di verificarti:

Fammi 3 domande per controllare se ho capito, poi correggimi le risposte.

3.2 Cosa consegni della Parte 2

1. Uno **screenshot** della risposta di Gemini.
2. Con parole tue: **cosa** ti ha spiegato e se hai dovuto **correggere** qualcosa (ricorda: l'IA può sbagliare).
3. Scrivi quale **versione** hai usato (Flash / Pro / ragionamento) e perché.

4. Parte 3 — NotebookLM con un PDF

4.1 Esercitazione

1. Nel browser entra su questo indirizzo con l'account della scuola:

notebooklm.google.com

2. Clicca **Crea nuovo**, poi **Carica**, e scegli il **PDF** indicato dal docente (per esempio la dispensa di teoria in PDF).
3. Aspetta che la fonte compaia a sinistra: è pronta.
4. Nel riquadro della chat scrivi:

Fammi un riassunto semplice di questo documento.

5. Poi chiedi:

Fammi 3 domande a risposta multipla su questo documento.

4.2 Cosa consegni della Parte 3

1. Uno **screenshot** del riassunto e delle domande generate.
2. Con parole tue: qual è la **differenza** che hai notato tra chiedere a **Gemini** e chiedere a **NotebookLM**?

5. Domanda finale (fa la differenza)

1. In tre righe, spiega il **flusso completo**: dove sta il materiale, cosa dai a Gemini, cosa carichi su NotebookLM, dove consegni. Usa parole tue.

6. Come sarai valutato (per sapere cosa conta)

1. **Parte 1 — Git/GitHub (30%)**: risposte corrette a parole tue e screenshot della storia dei commit.

2. **Parte 2 — Gemini con .md (25%):** esercitazione svolta, screenshot, e hai indicato la versione usata.
3. **Parte 3 — NotebookLM con PDF (25%):** esercitazione svolta, screenshot, e hai colto la differenza con Gemini.
4. **Domanda finale + parole tue (20%):** sai spiegare il flusso completo con parole tue (la prova del nove).

Conta piu il "saperlo spiegare a parole tue" che il copia-incolla: se sai raccontare cosa hai fatto, hai imparato davvero.

I sei lavori consegnati

Lavoro 1 (anonimo)

Teoria — Git, versioning e Intelligenza Artificiale (Gemini e NotebookLM)

Classe 4 · Informatica (Laboratorio Professionale) · Versione 1.0 · 29/09/2026

Documento di teoria per l'allievo, in versione testo (.md). E la stessa dispensa in formato testo puro: puoi darla a Gemini per farti spiegare o tradurre le parti difficili, o caricarla come fonte. I disegni (i grafi dei rami) qui sono descritti a parole. Regola d'oro del corso: l'IA serve a CAPIRE meglio, non a copiare; la prova del nove è saperlo spiegare a parole tue.

1. I tre strumenti e come si legano

Usiamo tre ambienti diversi, ognuno con un compito preciso:

Git e GitHub: il magazzino ordinato del lavoro, dove ogni versione resta salvata e condivisibile.

Gemini: l'IA (Intelligenza Artificiale) generativa a cui si parla; le si dà un testo (anche un file .md) e ti spiega, traduce, riassume.

NotebookLM: un tutor che risponde SOLO sui documenti che carichi (per esempio un PDF), quindi resta fedele alla fonte.

Idea da tenere a mente: Git conserva, Gemini spiega, NotebookLM ripassa sulla fonte. Non sono concorrenti: si usano insieme.

2. Git e GitHub

Git è un sistema di versionamento: tiene la storia del lavoro, così ogni salvataggio importante resta e si può tornare indietro.

GitHub è il sito in rete dove il lavoro versionato vive e si condivide: è "Git su internet". Attenzione a non confonderli: Git è lo strumento (il motore), GitHub è il servizio online (il garage online).

2.1 GitHub: un'azienda e un servizio

È un'azienda (GitHub, Inc., dal 2018 di Microsoft): come ogni azienda deve mantenersi.

È un servizio online: un sito dove metti i progetti in rete, li tieni al sicuro e ci lavori in gruppo, tutto da browser.

2.2 Il repository, il commit, il branch

Un repository (o "repo") è la cartella-progetto con dentro i file e tutta la loro storia.

Un commit è un salvataggio con etichetta: fotografa il progetto in quel momento e gli mette un messaggio ("cosa ho cambiato"). Tanti commit in fila fanno la storia.

Un branch (ramo) è una strada di lavoro parallela, per provare senza rovinare la versione principale (il main); riunire un ramo al main si chiama merge.

La Pull Request (PR) è la proposta di unire il proprio ramo nella versione principale, dopo che qualcuno l'ha controllata.

2.3 Che tipo di servizio: PaaS e Freemium

PaaS (Platform as a Service: piattaforma come servizio): qualcuno ti mette online una piattaforma già pronta e tu la usi senza costruirti server e computer. Fa parte della famiglia "...aaS" (IaaS = server nudi, PaaS = piattaforma pronta, SaaS = software pronto nel browser).

Freemium (free + premium): le funzioni di base sono gratis; per le funzioni avanzate si paga. A te studente basta la parte gratis.

3. Il versioning: le versioni 1.2.3

Quando un progetto raggiunge un punto "buono", gli si dà un numero di versione di tre cifre 1.2.3:

major (la prima cifra): aumenta quando cambi tutto in modo grosso, magari incompatibile col vecchio.

minor (la seconda): aumenta quando aggiungi una funzione nuova, ma il resto funziona ancora.

patch (la terza): aumenta quando correggi solo un bug, niente di nuovo.

Esempio: v1.0.0 prima versione giocabile, aggiungi i suoni v1.1.0, correggi un bug v1.1.1, rifai tutto da capo v2.0.0.

Una versione pubblicata (release) è "congelata".

[GIALLO] Regola d'oro del versioning: una versione già pubblicata non si riusa mai. Se cambi anche una sola cosa, prima aumenti il numero e poi pubblichi.

4. Casi d'uso di Git (i grafi, descritti a parole)

Sviluppo lineare: un solo autore, nessun ramo. I commit vanno in fila, la storia è una linea dritta. Se qualcosa si rompe, torni indietro di un commit.

Una funzione su un branch, poi merge: provi una funzione su un ramo separato mentre il main resta sano; quando è pronta la unisci (merge).

Due funzioni in parallelo: due rami partono insieme e si fondono in tempi diversi; il main raccoglie i pezzi man mano.

Hotfix urgente: durante una funzione lunga esce un bug grave; apri un ramo hotfix corto, lo correggi e lo unisci subito, poi torni alla tua funzione.

Le release: ogni tanto pubblichi una versione (bandierina sul main): v1.0, v1.1, v1.2... fino a un cambiamento grosso v2.0. Ogni release è congelata.

Ramo abbandonato: provi un'idea su un ramo e non la unisci mai; il main non ne risente. Nessun danno.

Ramo da un altro ramo: una funzione grande stacca un sotto-ramo; prima rientra il sotto-ramo nel ramo, poi il ramo nel main (scatole dentro scatole).

Grande integrazione (gruppo): ognuno ha il suo ramo (grafica, movimenti, audio, menu) e alla fine confluiscono tutti nel main con le Pull Request: nasce la v1.0.

Ramo di manutenzione: il main va verso la v2.0, ma per chi usa ancora la v1 tieni un ramo parallelo con solo i fix (v1.1, v1.2).

Revert: un commit ha introdotto un problema; non cancelli la storia, aggiungi un nuovo commit che annulla quello sbagliato (revert), lasciando traccia onesta.

Conflitto di merge: due rami cambiano la stessa riga in modo diverso; Git si ferma e una persona sceglie cosa tenere, chiudendo con un commit di risoluzione.

Squadra a 5 rami con due release: sul main si pubblicano v1.0 e v2.0 mentre quattro rami lavorano in parallelo e rientrano in momenti diversi.

Rami annidati a 5 livelli: un lavoro grande si divide in rami dentro rami, fino a cinque livelli; ogni livello rientra nel precedente, come le matrici.

Due major insieme: dopo la v1.0 il main punta alla v2.0, ma restano vivi due rami di manutenzione (v1.x e poi v2.x).

Storia realistica: tutto insieme — release, feature con sotto-ramo, hotfix, revert e un merge con conflitto: la fotografia di un progetto vero.

[GIALLO] Come si legge un grafo: ogni pallino e un commit, la linea orizzontale in alto e il main, le linee colorate che si staccano sono branch, quando due linee si riuniscono e un merge, le bandierine gialle sono le release.

5. I file Markdown (.md)

Un file Markdown (estensione .md) è testo puro con pochi simboli:

a inizio riga fa un titolo;

parola mette in grassetto;

una riga con 1. fa un elenco numerato.

Si legge ovunque e non contiene immagini pesanti: solo contenuto. Per questo l'IA lo legge molto bene: nel corso diamo le dispense anche in .md proprio per darle a Gemini.

6. Gemini: l'IA generativa, e come usarla con un file .md

Gemini è un'IA generativa: le scrivi una richiesta (un "prompt") e genera una risposta: spiega, traduce, riassume, fa domande di ripasso.

L'IA può sbagliare o inventare (allucinazioni): quello che dice va controllato, soprattutto date, numeri e fonti.

6.1 Passi per usare Gemini con un file .md

Nel browser vai su questo indirizzo, con l'account della scuola:

gemini.google.com

□

Apri il file .md, seleziona tutto e copialo; torna su Gemini e incolla (oppure allega il file).

Scrivi la richiesta, per esempio:

□ Spiegami con parole semplici la parte che non capisco: [incolla la frase].

□ 6.2 Quale versione di Gemini scegliere

In alto Gemini ha un menu per la versione del modello. I nomi/numeri cambiano; conta a cosa serve ciascuna:

Flash (veloce): risposte rapide per cose semplici (spiegazioni, traduzioni, riassunti brevi). La scelta di tutti i giorni.

Pro (per esempio "3.1 Pro", più potente): per compiti complessi e testi lunghi. Più lenta ma più capace.

Ragionamento (per esempio "3.6 ragionamento", che "pensa" prima di rispondere): per problemi difficili, passo per passo. La più lenta ma la più accurata.

Regola pratica: parti da Flash; se la risposta è complessa o ti sembra sbagliata, passa a Pro o Ragionamento e richiedi.

Guarda a cosa serve la versione (veloce / potente / ragiona), non solo al numero.

[GIALLO] Le versioni più potenti o di "ragionamento", con l'account gratuito, possono avere un limite giornaliero: usale quando servono davvero, per il resto va bene Flash.

7. NotebookLM: il tutor sulle fonti, e come usarlo con un PDF

NotebookLM risponde solo sui documenti che carichi: se carichi un PDF, ti risponde usando quel PDF e non inventa fuori da lì.

È ottimo per studiare e ripassare su una fonte precisa (una dispensa, un capitolo): resta fedele al testo.

7.1 Passi per usare NotebookLM con un PDF

Nel browser vai su questo indirizzo, con l'account della scuola:

notebooklm.google.com

□

Clicca Crea nuovo, poi Carica, e scegli un file PDF; aspetta che la fonte compaia a sinistra.

Nel riquadro della chat scrivi, per esempio:

□ Fammi un riassunto semplice di questo documento.

□

[GIALLO] Se non hai un PDF sul computer: su github.com entra nella cartella del documento, clicca sul PDF e poi sull'icona di download (freccia in giù) in alto a destra.

8. Il flusso completo: i tre ambienti insieme

Il materiale del corso sta su Git/GitHub (versionato).

Da lì si scarica il PDF (per leggere/stampare) o il file .md (per l'IA).

Il .md si dà a Gemini per farsi spiegare o tradurre.

Il PDF si carica su NotebookLM per interrogarlo e ripassare restando sulla fonte.

Il lavoro finito si consegna su Classroom.

Strumento

A cosa serve

Cosa gli dai

Git / GitHub

Conservare e condividere il lavoro versionato

I file del progetto

Gemini

Spiegare, tradurre, riassumere

Un testo o un file .md

NotebookLM

Ripassare restando fedele alla fonte

Un documento (per esempio un PDF)

9. Uso responsabile dell'IA

Capire, non copiare: l'IA è un aiuto per imparare, non un modo per saltare il lavoro. La prova del nove: saperlo spiegare a parole proprie.

Controllare sempre: l'IA può sbagliare o inventare; verifica quello che dice.

Privacy e dati: non inserire dati personali, di compagni o di famiglia negli assistenti.

10. Prendi appunti a mano

Prima di usare gli strumenti, disegna sul quaderno tre riquadri (Git, Gemini, NotebookLM) e, per ognuno, a cosa serve e cosa gli dai.

Gli appunti a mano confluiscono nel tuo quaderno personale: aiutano a fissare i concetti e servono per la prova del nove.

Changelog

v1.0 (29/09/2026): prima versione. Git/GitHub, versioning e 15 casi d'uso (a parole), file Markdown, Gemini con .md e le sue versioni (Flash/Pro/ragionamento), NotebookLM con PDF, flusso completo e uso responsabile.

Lavoro 2 (anonimo)

PARTE 1

- 1) Git è lo strumento (la tecnologia che tiene la storia di un progetto, gratis e di nessuno); GitHub è il servizio online (l'azienda) dove quei progetti vivono in rete. Git è il motore, GitHub è il garage online.
- 2) Un repository (in breve "repo") è la cartella-progetto con dentro i file e tutta la loro storia.
- 3) Ogni salvataggio della storia si chiama commit (una "fotografia" del progetto con un messaggio).
- 4) Una strada di lavoro parallela che puoi aprire se lavori in gruppo. Una Pull Request propone di unire le modifiche fatte su un ramo parallelo (branch) all'interno di un altro ramo, di solito quello principale (main).

Lavoro 3 (anonimo)

1. I tre strumenti e la loro integrazione

Git e GitHub (Conservare): Git è il motore locale per il versionamento del codice; GitHub è il servizio cloud (PaaS/Freemium) dove salvare e condividere il lavoro.

Gemini (Spiegare): IA generativa ideale da usare con file .md (Markdown). Serve per chiarire concetti, tradurre e riassumere. Offre modelli differenti: Flash (veloce per l'uso quotidiano), Pro (per testi complessi) e Ragionamento (per problemi logici).

NotebookLM (Ripassare): Tutor IA basato su fonti. Risponde esclusivamente sui file caricati (es. PDF), evitando di inventare o cercare informazioni esterne.

2. Versionamento e concetti di Git

Elementi chiave:

Repository: la cartella del progetto contenente tutti i file e la loro cronologia.

Commit: salvataggio puntuale con etichetta che descrive la modifica effettuata.

Branch: ramo di lavoro parallelo per testare modifiche senza rovinare il ramo principale (main).

Merge: operazione di unione tra due rami.

Pull Request (PR): richiesta formale di revisione per unire il proprio ramo al main.

Versioning Semantico (MAJOR.MINOR.PATCH):

Major: modifiche di grande portata o incompatibili con le versioni precedenti.

Minor: introduzione di nuove funzionalità retrocompatibili.

Patch: semplici correzioni di bug.

Regola d'oro: Una versione pubblicata è congelata. Per qualsiasi modifica successiva si deve prima incrementare il numero di versione.

3. Scenari di utilizzo di Git

Git gestisce diversi flussi di lavoro, tra cui:

Sviluppo lineare: commit in sequenza su un solo ramo.

Rami Feature e Hotfix: creazione di rami per nuove funzionalità o correzioni urgenti di bug critici, poi riaccorpati via merge.

Revert: annullamento di un commit errato aggiungendo un nuovo commit, preservando la storia del progetto.

Gestione dei conflitti: intervento manuale richiesto quando due rami modificano la stessa riga di testo in modo divergente.

4. Markdown e uso responsabile delle tecnologie

Markdown (.md): formato di testo puro con marcatori essenziali (#, **), ideale per essere elaborato velocemente dai modelli di IA.

Principi guida: L'IA deve servire a comprendere e non a copiare. È fondamentale verificare sempre la correttezza dei dati generati, proteggere la propria privacy non inserendo dati personali e saper spiegare i concetti a parole proprie.

Differenza tra Git e GitHub

Git è lo strumento che usi sul tuo computer (il "motore") per registrare e gestire le varie versioni dei tuoi file.

GitHub è il sito internet (il "garage online") dove carichi il lavoro fatto con Git per salvarlo sul cloud e condividerlo con gli altri.

2) Che cos'è un repository

Un repository (o "repo") è la cartella del tuo progetto che contiene sia i file su cui stai lavorando, sia tutta la loro cronologia di salvataggi.

3) Cos'è un commit e a cosa serve la storia

Un commit è una "fotografia" istantanea del progetto in un dato momento, a cui si aggiunge una breve nota descrittiva (es. "aggiunto menu principale").

La storia dei commit serve a capire come è cambiato il progetto nel tempo e permette di tornare indietro a una versione precedente se si verifica un errore.

4) A cosa servono Branch e Pull Request

Un branch (ramo) è una corsia di lavoro parallela che ti permette di provare nuove idee o correggere errori senza rovinare la versione principale e funzionante del progetto (main).

Una Pull Request (PR) è la richiesta di unire il tuo ramo alla versione principale, consentendo ad altri di controllare il codice prima di integrarlo definitivamente.

Compito: Git, Versioning e Intelligenza Artificiale (Gemini e NotebookLM)

Classe: 4^a Informatica (Laboratorio Professionale)

Studente: [Cognome e Nome]

Data: 29 Settembre 2026

Modulo: Git, GitHub, Gemini e NotebookLM

Parte 1 — Git e GitHub

1.1 Risposte alle domande teoriche

1. Con parole tue, che differenza c'è tra Git e GitHub?

Git è il programma (il motore di versione) installato sul proprio computer che si occupa di salvare, tracciare e gestire la cronologia di tutte le modifiche fatte ai file di un progetto. GitHub, invece, è il servizio online sul cloud (il "garage" o "magazzino" online) in cui vengono caricati i progetti gestiti con Git per poterli salvare in remoto, condividere e sviluppare in gruppo tramite browser.

2. Che cos'è un repository?

Un repository (abbreviato in "repo") è la cartella principale di un progetto, all'interno della quale si trovano tutti i file di lavoro insieme a tutto lo storico salvato delle modifiche fatte nel tempo.

3. Che cos'è un commit e a cosa serve avere la storia dei commit?

Un commit è una vera e propria "fotografia" istantanea del progetto in un determinato momento, associata a un messaggio esplicativo che descrive cosa è stato cambiato. Avere la storia dei commit serve a comprendere come il progetto si è evoluto nel tempo, chi ha fatto cosa e consente di tornare indietro a una versione precedente e funzionante qualora si verifichi un errore o un problema nel codice.

4. A cosa serve un branch e cosa propone una Pull Request?

Un branch (ramo) è una linea di lavoro separata dalla versione principale (main) che permette di sviluppare nuove funzionalità o fare esperimenti senza rischiare di rovinare il lavoro stabile. Una Pull Request (PR) è una proposta formale con cui si chiede ai compagni di squadra o al docente di revisionare il codice scritto sul proprio ramo e di approvarne l'unione (merge) all'interno della versione principale.

1.2 Azione pratica su GitHub

Di seguito viene riportata la prova dello svolgimento dell'esercizio pratico svolto nel repository del corso su github.com:

[AREA RISERVATA ALLO SCREENSHOT DELLA STORIA DEI COMMIT]

Per inserire la cattura schermo: premi Windows + Maiusc + S, seleziona la cronologia dei commit su GitHub e poi incolla qui con Ctrl + V.

Descrizione dell'ultimo commit osservato:

Dall'analisi della cronologia dei commit nell'interfaccia di GitHub, la schermata mostra la sequenza cronologica dei salvataggi del repository. L'ultimo commit (versione v1.0) indica l'aggiornamento della dispensa teorica con la revisione dei 15 casi d'uso di Git e la descrizione dei modelli di Intelligenza Artificiale (Gemini e NotebookLM).

Parte 2 — Gemini con un file .md

2.1 Esercitazione e spiegazione dei concetti

Che cosa sono i file Markdown (.md) e perché sono comodi per l'IA?

I file Markdown (con estensione .md) sono documenti scritti in testo puro che utilizzano una formattazione leggermente marcata con pochi simboli semplici (ad esempio # per i titoli e ** per il grassetto). Sono estremamente comodi per l'Intelligenza Artificiale perché privi di pesantezza grafica o codici complessi di impaginazione: l'IA può leggere ed elaborare direttamente il testo senza spreco di risorse, facilitando spiegazioni, riassunti e traduzioni dirette.

Domande di verifica generate da Gemini e risposte svolte:

Domanda 1: Qual è la differenza tra i tre numeri del versionamento semantico (es. 1.2.3)? Risposta: Il primo numero è la major (modifiche strutturali o incompatibili), il secondo è la minor (nuove funzioni compatibili) e il terzo è la patch (correzione di bug).

Domanda 2: Cosa succede se modifichi un solo file in una versione già pubblicata? Risposta: Una versione pubblicata è congelata e non si modifica mai. Occorre prima incrementare il numero di versione e poi pubblicare la nuova release.

Domanda 3: Perché si crea un ramo (branch) prima di sviluppare una nuova funzione?

Risposta: Per lavorare in modo isolato senza rischiare di danneggiare la versione principale (main) fino a quando la nuova funzione non è pronta e verificata.

2.2 Resoconto per la consegna

[AREA RISERVATA ALLO SCREENSHOT DELLA RISPOSTA DI GEMINI]

Incolla qui lo screenshot dell'interazione avvenuta con Gemini su gemini.google.com.

Spiegazione ricevuta e verifica della correttezza:

Gemini ha spiegato la struttura dei file Markdown evidenziando la loro semplicità rispetto ai classici documenti Word. Durante l'esercitazione le risposte fornite dall'IA sono risultate accurate; tuttavia è sempre necessario verificare con attenzione le definizioni teoriche poiché i modelli generativi potrebbero generare inesattezze (allucinazioni).

Versione di Gemini utilizzata:

Per questo compito è stata utilizzata la versione Gemini Flash. Questa versione è stata scelta perché è veloce, efficiente ed è la soluzione ideale per le attività quotidiane di spiegazione, sintesi e riassunto di file di testo.

Parte 3 — NotebookLM con un PDF

3.1 Esercitazione sul documento PDF

Riassunto semplice generato da NotebookLM sulla dispensa:

Il documento illustra l'integrazione di tre ambienti fondamentali per lo studente d'informatica: Git/GitHub per la conservazione e il versionamento del codice, Gemini per spiegare e chiarire i concetti scritti in Markdown, e NotebookLM per ripassare ed effettuare test di verifica ancorati esclusivamente al documento PDF caricato.

Domande a risposta multipla generate da NotebookLM:

Qual è il compito principale di NotebookLM secondo la dispensa?A) Scrivere codice da zeroB) Rispondere esclusivamente sui documenti caricati per ripassare sulla fonteC) Sostituire il servizio di hosting GitHubRisposta corretta: B

Cosa rappresenta la terza cifra nel versionamento semantico (es. v1.2.3)?A) MajorB) MinorC) PatchRisposta corretta: C

Come si chiama la proposta di unire il proprio ramo alla versione principale?A) CommitB) Pull Request (PR)C) RevertRisposta corretta: B

3.2 Resoconto per la consegna

[AREA RISERVATA ALLO SCREENSHOT DI NOTEBOOKLM]

Incolla qui lo screenshot con il riassunto e i quiz generati da NotebookLM su notebooklm.google.com.

Differenza tra Gemini e NotebookLM:

Gemini è un'IA generativa ad ampio spettro che attinge a un'ampia conoscenza generale ed è ottima per spiegare, riformulare e tradurre concetti liberamente. Al contrario, NotebookLM è un tutor rigorosamente "ancorato" alle fonti: lavora solo ed

esclusivamente sui documenti caricati dall'utente (es. PDF) e garantisce risposte fedeli alla fonte senza inventare o recuperare informazioni esterne.

Parte 4 — Domanda Finale: Il Flusso Completo

Il materiale di studio e i progetti risiedono in modo ordinato e versionato su Git/GitHub. Da qui si preleva il file .md per farlo spiegare e chiarire a Gemini, mentre si carica il file PDF su NotebookLM per interrogare la fonte ed effettuare il ripasso guidato. Una volta compresi i concetti e completate le attività, la consegna finale del lavoro si effettua su Google Classroom.

Lavoro 4 (anonimo)

Git è lo strumento che tiene le source code dei tuoi programmi. Invece GitHub è il servizio online dove i tuoi progetti sono pubblici.

Il repository è una cartella di progetto online. Contiene tutti i file del codice, la storia delle modifiche e permette a più persone di lavorarci insieme senza sovrascrivere il lavoro altrui.

Un commit è un salvataggio del progetto.

Lavoro 5 (anonimo)

Sintesi delle Tecnologie e Integrazione

Response: Ecco una sintesi chiara del documento sulle tre tecnologie e la loro integrazione:

1. I tre strumenti principali

Git e GitHub: Servono per conservare e versionare il lavoro. Git è il motore che tiene traccia della storia dei file, mentre GitHub è la piattaforma online su cui condividere i progetti e lavorare in gruppo.

Gemini: È un'intelligenza artificiale generativa versatile. La si usa passandole testi o file Markdown (.md) per spiegare, tradurre, riassumere o fare esercitazioni di ripasso.

NotebookLM: È un assistente virtuale focalizzato. Risponde e genera contenuti basandosi unicamente sui documenti specifici caricati dall'utente (es. PDF), garantendo fedeltà alla fonte ed evitando invenzioni.

2. Concetti chiave su Git e Versionamento

Componenti base: Un repository è la cartella del progetto; un commit è un salvataggio con messaggio della storia; un branch è un ramo di lavoro autonomo; la Pull Request è la richiesta di unire un ramo al ramo principale (main).

Numerazione delle versioni (major.minor.patch):

major: cambiamenti grandi e potenzialmente incompatibili.

minor: aggiunta di nuove funzionalità.

patch: correzione di bug.

Regola d'oro: Una versione già pubblicata non si modifica mai; per qualsiasi modifica occorre incrementare il numero di versione prima di una nuova pubblicazione.

3. Modelli di Gemini e selezione dell'uso

Flash: Modello rapido per le esigenze quotidiane (spiegazioni semplici, riassunti brevi, traduzioni).

Pro: Modello più potente ideato per testi complessi e compiti impegnativi.

Ragionamento: Modello che analizza il problema passo dopo passo, ideale per compiti di logica complessi o che richiedono estrema accuratezza.

4. Flusso di lavoro integrato

Strumento

A cosa serve

Cosa si fornisce in input

Git / GitHub

Conservare e condividere il lavoro versionato

File del progetto

Gemini

Spiegare, tradurre e riassumere liberamente

Testo o file .md

29/09/26, 12:42

Sintesi delle Tecnologie e Integrazione

1/2

Strumento

A cosa serve

Cosa si fornisce in input

NotebookLM

Ripassare e fare domande restando fedeli alla
fonte

Documenti ufficiali (es.
PDF)

Nota di utilizzo responsabile: L'IA è uno strumento da utilizzare per comprendere i concetti, non per sostituire il lavoro individuale. La verifica finale dell'apprendimento risiede sempre nella capacità di spiegare i concetti a parole proprie.

29/09/26, 12:42

Sintesi delle Tecnologie e Integrazione

2/2

Lavoro 6 (anonimo)

2. Parte 1 — Git e GitHub

2.1 Domande (rispondi a parole tue)

1. Con parole tue, che differenza c'è tra Git e GitHub?

allora Git è lo strumento che usi per gestire la cronologia dei tuoi file in locale sul tuo PC, mentre GitHub è la casa online dove pubblichi e condividi quei progetti con gli altri.

2. Che cos'è un repository?

Un repository è la cartella del tuo progetto gestita da Git. A differenza di una normale cartella del computer, un repository salva automaticamente tutte le modifiche fatte nel tempo ai file. Questo ti permette di vedere cosa è stato cambiato, da chi è stata fatta la modifica e, se necessario, di recuperare e ripristinare qualsiasi versione precedente del lavoro.

3. Che cos'è un commit e a cosa serve avere la storia dei commit?

Un commit è un salvataggio manuale dello stato dei tuoi file in un determinato momento. Quando fai un commit, scatti una "fotografia" esatta di come si presenta il tuo progetto in quell'istante, accompagnata da un breve messaggio esplicativo che descrive cosa hai modificato.

4. A cosa serve un branch e cosa propone una Pull Request?

Ti permette di fare esperimenti e modifiche in totale sicurezza. Se l'esperimento va male, puoi semplicemente eliminare il branch senza aver toccato il progetto principale.